

DEEP LEARNING PROJECT FASHION MNIST CLASSIFICATION

INDEX:

1.ABSTRACT

2.OBJECTIVE

3.INTRODUCTION

4.METHODOLOGY

5.CODE

6.CONCLUSION

1.ABSTRACT:

In this project, I have built a fashion apparel recognition using the Convolutional Neural Network (CNN) model. To train the CNN model, we have used the Fashion MNIST dataset. After successful training, the CNN model can predict the name of the class given apparel item belongs to. This is a multiclass classification problem in which there are 10 apparel classes the items will be classified.

The fashion training set consists of 70,000 images divided into 60,000 training and 10,000 testing samples. Dataset sample consists of 28x28 grayscale images, associated with a label from 10 classes.

So the end goal is to train and test the model using Convolution neural network.

2.OBJECTIVE:

This work is part of my experiments with Fashion-MNIST dataset using various Machine Learning algorithms/models. The objective is to identify (predict) different fashion products from the given images using various best possible Machine Learning Models (Algorithms) and compare their results (performance measures/scores) to arrive at the best ML model. I have also experimented with 'dimensionality reduction' technique for this problem.

Artificial Intelligence is among the next big things in the software engineering field that empowers numerous applications in health care, finance, logistic, industries for good measure. Several scientists have embarked on utilising the advancement of Artificial Intelligence and high-grade dataset to change our lives profoundly. One notable field in Artificial Intelligence is Computer Vision, which rapidly emerged over the last decade thank to an enormous amount of visual data and significant development in GPU processing power. Neural networks can now have millions of trainable parameters which makes technologies like diseases self-diagnoses or self-driving car possible.

In this work, we implemented a multi-layer neural network to classify images of fashion products using the Zalando's Fashion MNIST Dataset using softmax outputs for classification in order to identify strategies that optimize performance in neural networks. The multi-layer neural network included inputs, hidden layers, and outputs. The goal was to classify 28x28 greyscale images into one of 10 classes: top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, and ankle boot. The Fashion MNIST dataset is a more complicated problem to solve compared to the classical MNIST dataset.

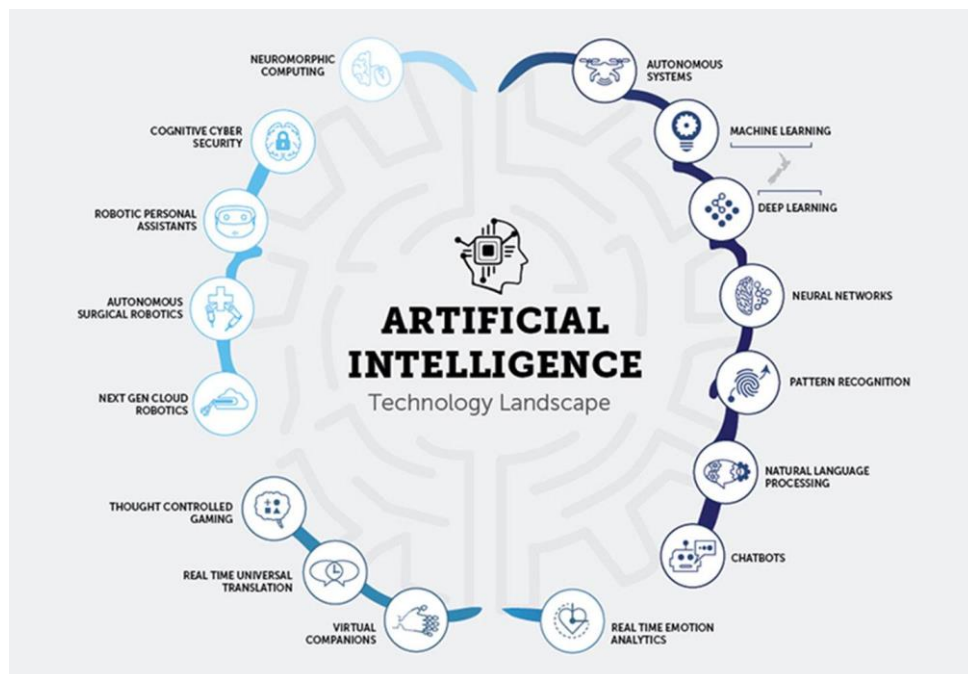
First, a neural network was implemented containing 2 hidden layers, each with 50 units, and tanh activation for the hidden layers. Cross-entropy loss was used as the objective function. We used mini-batch stochastic gradient descent for training throughout. Here we compared different learning rates and minibatch sizes to find the best model. Next, we performed various experiments on the multi-layer neural network with hopes of optimizing the performance of the network.

We experimented with regularization where we added a weight decay, L2 regularization, in the update scheme. Additionally, we tested various activation functions for the hidden units to compare to the initial tanh function that was used. Lastly, we varied the neural network topology. In each scenario, we analyzed network performance in order to optimize classification and minimize loss.

Figure 1 summarizes the results from our network optimization experiments. The rest of this work goes into the findings and the resulting intuition gleaned in detail.

An overview of Deep Learning:

Artificial Intelligence is a vast field which concerns several areas like Statistic, Optimization, Machine Learning and Deep Learning. This section introduces DL as the state-of-the-art solution for AI problems with complex datasets like visual data, sound or natural languages and how it's related other fields of AI. Besides, this section explains the concepts of Convolutional Neural Network, optimization process of Neural Network training and then dives deeper to Mathematics theory behind their implementation.



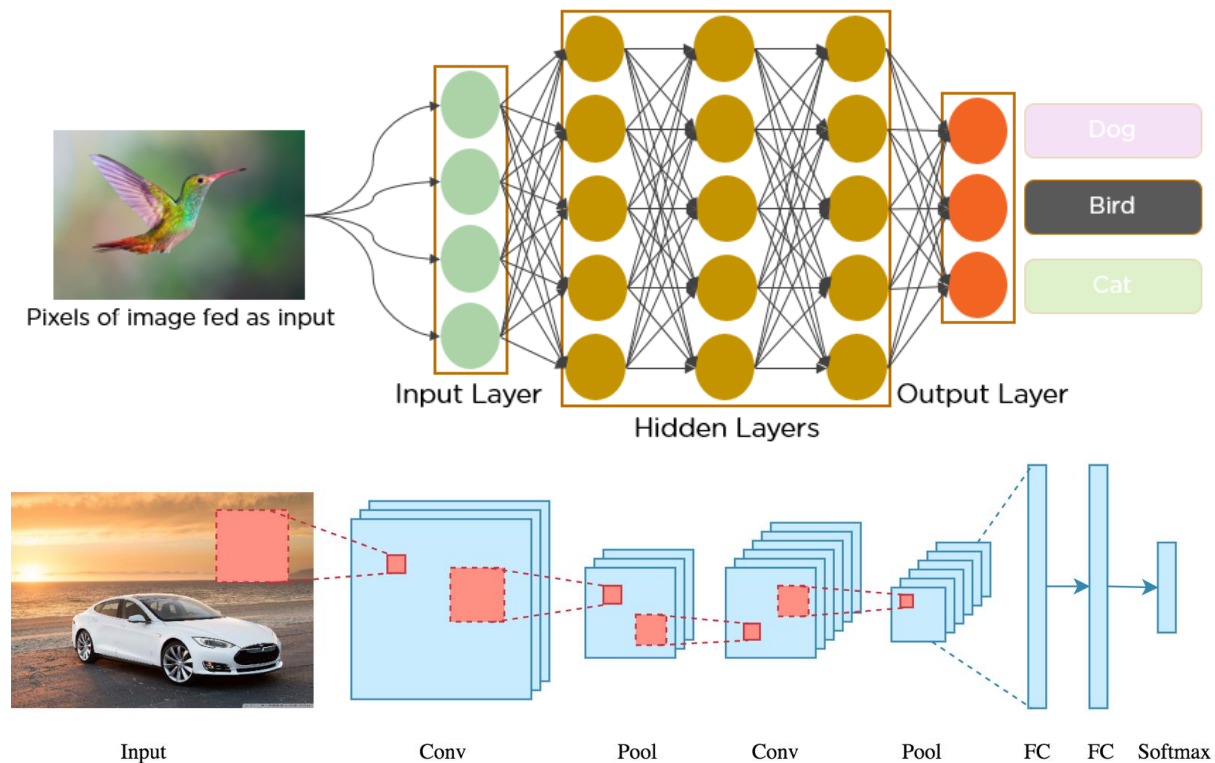
3.INTRODUCTION:

A tremendous interest in deep learning has emerged in recent years . The most established algorithm among various deep learning models is convolutional neural network (CNN), a class of artificial neural networks that has been a dominant method in computer vision tasks since the astonishing results were shared on the object recognition competition known as the ImageNet Large Scale Visual Recognition Competition (ILSVRC) in 2012. Medical research is no exception, as CNN has achieved expert-level performances in various fields. Gulshan et al, Esteva et al. , and Ehteshami Bejnordi et al demonstrated the potential of deep learning for diabetic retinopathy screening, skin lesion classification, and lymph node metastasis detection, respectively. Needless to say, there has been a surge of interest in the potential of CNN among radiology researchers, and several studies have already been published in areas such as lesion detection , classification , segmentation , image reconstruction , and natural language processing. Familiarity with this state-of-the-art methodology would help not only researchers who apply CNN to their tasks in radiology and medical imaging, but also clinical radiologists, as deep learning may influence their practice in the near future. This article focuses on the basic concepts of CNN and their application to various radiology tasks, and discusses its challenges and future directions.

What is CNN ?

CNN is a type of deep learning model for processing data that has a grid pattern, such as images, which is inspired by the organization of animal visual cortex and designed to automatically and adaptively learn spatial hierarchies of features, from low- to high-level patterns. CNN is a mathematical construct that is typically composed of three types of layers (or building blocks): convolution, pooling, and fully connected layers. The first two, convolution and pooling layers, perform feature extraction, whereas the third, a fully connected layer, maps the extracted features into final output, such as classification. A convolution layer plays a key role in CNN, which is composed of a stack of mathematical operations, such as convolution, a specialized type of linear operation. In digital images, pixel values are stored in a two-dimensional (2D) grid, i.e., an array of numbers , and a small grid of parameters called kernel, an optimizable feature extractor, is applied at each image position, which makes CNNs highly efficient for image processing, since a feature may occur anywhere in the image. As one layer feeds its output into the next layer,

extracted features can hierarchically and progressively become more complex. The process of optimizing parameters such as kernels is called training, which is performed so as to minimize the difference between outputs and ground truth labels through an optimization algorithm called backpropagation and gradient descent, among others.

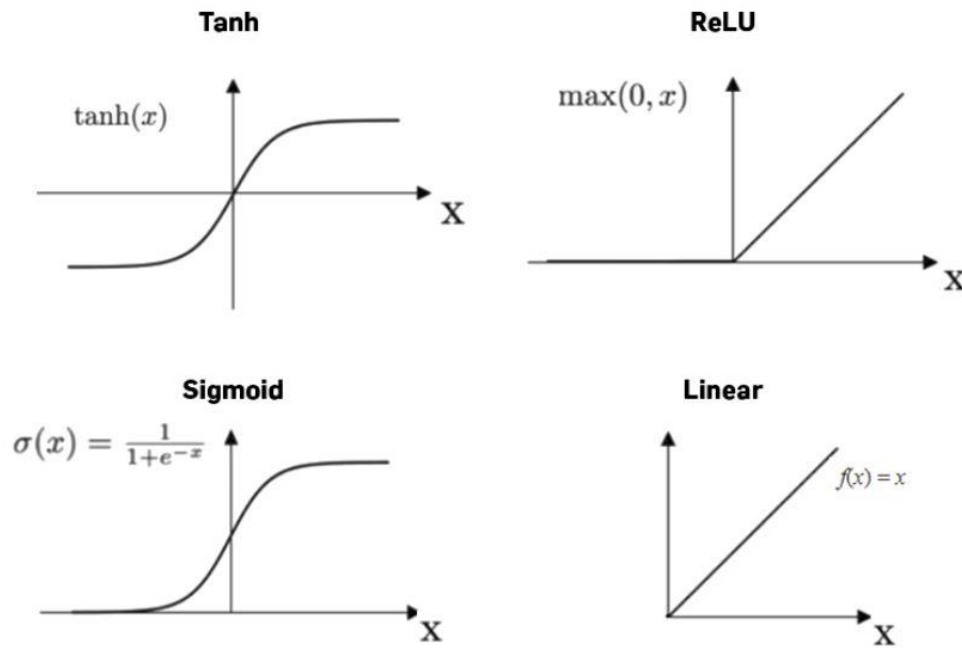


Convolution:

A Convolutional Neural Network (ConvNet/CNN) is a Deep Learning algorithm which can take in an input image, assign importance (learnable weights and biases) to various aspects/objects in the image and be able to differentiate one from the other. The pre-processing required in a ConvNet is much lower as compared to other classification algorithms. While in primitive methods filters are hand-engineered, with enough training, ConvNets have the ability to learn these filters/characteristics.

Nonlinear activation function:

The outputs of a linear operations such as convolution are then passed through a nonlinear activation function. Although smooth nonlinear functions, such as sigmoid or hyperbolic tangent (tanh) function, were used previously because they are mathematical representation of a biological neuron behaviour, the most common nonlinear unit (ReLU), which simply computes the function: $f(x) = \max(0, x)$.



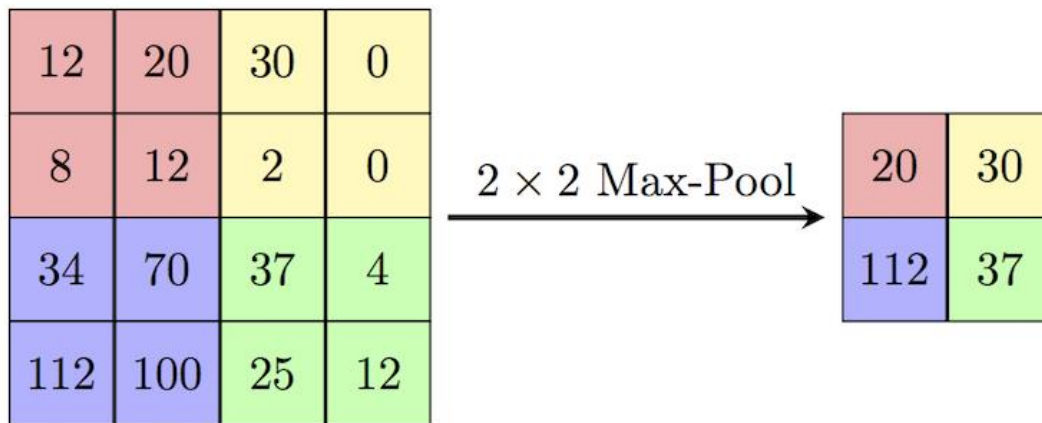
Activation functions commonly applied to neural networks: a rectified linear unit (ReLU), b sigmoid, and c hyperbolic tangent (tanh).

Pooling Layer:

A pooling layer provides a typical down sampling operation which reduces the in-plane dimensionality of the feature maps in order to introduce a translation invariance to small shifts and distortions, and decrease the number of subsequent learnable parameters. It is of note that there is no learnable parameter in any of the pooling layers, whereas filter size, stride and padding are hyperparameters in pooling operations, similar to convolution operations.

Max pooling:

The most popular form of pooling operation is max pooling, which extracts patches from the input feature maps, outputs the maximum value in each patch, and discards all the other values. A max pooling with a filter size 2×2 with a stride of 2 is commonly used in practice. This downsamples the in-plane dimension of feature maps by factor of 2. Unlike height and width, the depth dimension of feature maps remains unchanged.



Global average pooling:

Another pooling operation worth noting is a global average pooling [20]. A global average pooling performs an extreme type of downsampling, where a feature map with size of $height \times width$ is downsampled into a 1×1 array by simply taking the average of all the elements in each feature map, whereas the depth of feature maps is retained. This operation is typically applied only once before the fully connected layers. The advantages of applying global average pooling are as follows: (1) reduces the number of learnable parameters and (2) enables the CNN to accept inputs of variable size.

Fully connected layer:

The output feature maps of the final convolution or pooling layer is typically flattened, i.e., transformed into a one-dimensional (1D) array of numbers (or vector), and connected to one or more fully connected layers, also known as dense layers, in which every input is connected to every output by a learnable weight. Once the features extracted by the convolution layers and downsampled by the pooling layers are created, they are mapped by a subset of fully connected layers to the final outputs of the network, such as the probabilities for each class in classification tasks. The final fully connected layer typically has the same number of output nodes as the number of classes. Each fully connected layer is followed by a nonlinear function, such as ReLU, as described above.

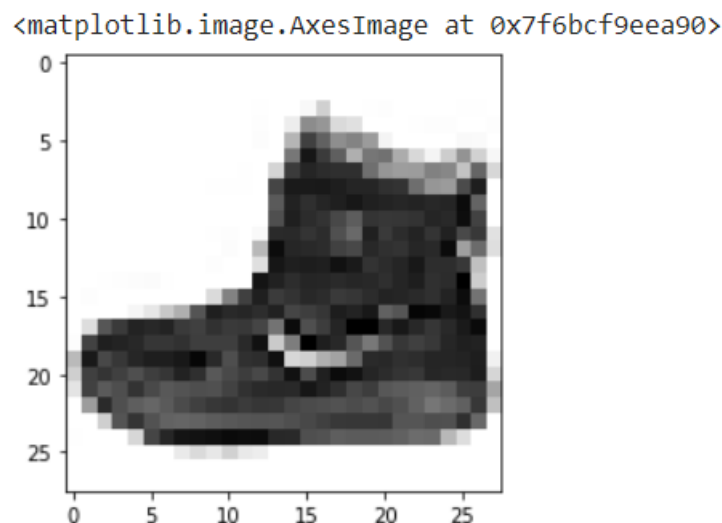
Last layer activation function:

The activation function applied to the last fully connected layer is usually different from the others. An appropriate activation function needs to be selected according to each task. An activation function applied to the multiclass classification task is a softmax function which normalizes output real values from the last fully connected layer to target class probabilities, where each value ranges between 0 and 1 and all values sum to 1. Typical choices of the last layer activation function for various types of tasks are summarized.

Fashion MNIST Clothing Classification:

The Fashion-MNIST dataset is proposed as a more challenging replacement dataset for the MNIST dataset. It is a dataset comprised of 60,000 small square 28×28 pixel grayscale images of items of 10 types of clothing, such as shoes, t-shirts, dresses, and more. The mapping of all 0-9 integers to class labels is listed below.

- 0: T-shirt/top
- 1: Trouser
- 2: Pullover
- 3: Dress
- 4: Coat
- 5: Sandal
- 6: Shirt
- 7: Sneaker
- 8: Bag
- 9: Ankle boot



4.METHODOLOGY:

Images were loaded and preprocessed from the provided Fashion MNIST dataset. Each image, originally 28x28 pixels, was flattened to a 1D vector of pixels of length 784 and normalized. The images for each category were then stacked into a data matrix with dimensions of the number of examples by the flattened number of pixels. Output labels were transformed using one hot encoding. The training data was loaded from the provided file and split 80/20 into a training and validation set. The testing data was used as provided. The neural network parameters were loaded from a config file and implemented as detailed below.

4.1 Multi-Layer Neural Network Implementation:

The multi-layer neural network was originally built with the specifications provided in the assignment. We used an input layer, two hidden layers each with 50 units and tanh activation, and a softmax output layer. The neural network comprised of three classes: Activation, Layer, and Neuralnetwork. The Activation class defined all the activation functions and their gradients as well as forward and backward pass functions. The activation functions were treated as an additional layer on top of a linear layer that computed the weight sum of inputs to that unit. The Layer class was for standard linear layers with both forward and backward passes. The forward function converted the input vector "x" to an output variable "a" to then be passed to the activation function. The backward function used the weighted sum of deltas and computed the weight and bias gradients. For each layer, weights and bias were initialized according to normal distributions with zero mean and unit standard deviation. They were updated by gradient descent with momentum and L2 regularization. Parameters (inputs, outputs, weights and gradients) for every layer were stored here. Lastly, the Neuralnetwork class created a list of Layers. Forward and backward functions defined in this class recursively call the forward and backward functions in Activation and Layer classes. Loss function and initial delta were normalized by number of classification classes (10 for Fashion MNIST) and number of examples in each mini-batch. This class was initialized with specifications given in the configuration file. These three classes were used for later tasks: numerical approximation, training, testing and experimentation. For training the neural network, we used mini-batch stochastic gradient descent as

well as momentum in our update rule weighted by a term γ . We also implemented early stopping to use the weights 2 with the best performance on the validation set during the training period; however, for the purposes of generating graphs for this report, we allowed the model to run for the full number of epochs and stored the weights that best performed on the validation set. We used an initial learning rate of $1.0e - 3$, mini-batch size of 128, number of epochs of 100, and momentum gamma of 0.9. After trying other values for these features in order to optimize the model, we found that changing the learning rate of $1.2e - 2$ while leaving the other parameters the same gave the best performance.

4.2 Numerical Approximation:

In order to confirm that our code was working, we computed the gradients using numerical approximation: $\frac{d}{dw} E(w) \approx \frac{E(w + \epsilon) - E(w - \epsilon)}{2\epsilon}$ (1) with $\epsilon = 10^{-2}$ and compared the results with answers obtained from backpropagation. We used 10 examples, one in each category, and added 10 numbers in the end for comparison. Each final answer is computed with only one weight changed in the network for numerical approximation.

4.3 Network Optimization Strategies:

4.3.1 Regularization:-

In order to further optimize our network, we experimented with a variety of aspects of the model. We initially implemented L2 (ridge) regularization, which works to penalize large weights by adding a sum of the weights squared to the loss term. This influences the weight update rule by adding a $2W$ term weighted by a regularization factor, 2λ . We varied the regularization term to optimize the model performance.

4.3.2 Activation Functions:

Next, using the best performing regularization rate, we implemented various different hidden unit activation functions to compare to the loss and accuracy obtained by the tanh function. These included sigmoid, ReLU, and leakyReLU activation functions. In order to obtain good performance and avoid overflow errors with these activation functions, we decreased the learning rate by 2 orders of magnitude ($1.2e-4$) and decreased the batch size to one, leading to slower training.

4.3.3 Network Topology:

Lastly, we experimented by changing the neural network topology. This involved halving and doubling the number of hidden units in each layer to analyze the effect on performance. In addition to 50×50 , we used the combinations of 50×100 , 100×50 , 25×50 , and 50×25 . We also changed the number of hidden layer to 1 and 3 instead of 2 while maintaining approximately the same number of parameters. This means we used 53 nodes in the hidden layer for the network with one hidden layer and 47 nodes in each layer in the 3 layer network.

5.CODE:

Fashion MNIST Data Classification Project:

Step-1: Import Libraries:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import keras
```

Step-2: Load Data:

```
(X_train, y_train), (X_test, y_test)=tf.keras.datasets.fashion_mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 [=====] - 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 [=====] - 0s 0us/step
```

print the shape of data

```
[ ] X_train.shape,y_train.shape,"*****",X_test.shape,y_test.shape  
  
((60000, 28, 28), (60000,)), '*****', (10000, 28, 28), (10000,))
```

▶ X_train[0]

```
array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0,  0],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  
        0,  0, 13, 73,  0,  0,  1,  4,  0,  0,  0,  0,  1,  
        1,  0],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  3,  
        0, 36, 136, 127, 62, 54,  0,  0,  0,  1,  3,  4,  0,  
        0,  3],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  6,  
        0, 102, 204, 176, 134, 144, 123, 23,  0,  0,  0,  0, 12,  
       10,  0],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  
        0, 155, 236, 207, 178, 107, 156, 161, 109, 64, 23, 77, 130,  
       72, 15],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  0,  
        69, 207, 223, 218, 216, 216, 163, 127, 121, 122, 146, 141, 88,  
       172, 66],  
       [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  1,  1,  0,
```



```
[ 0, 0, 0, 0, 18, 44, 82, 107, 189, 228, 220, 222, 217,
 226, 200, 205, 211, 230, 224, 234, 176, 188, 250, 248, 233, 238,
 215, 0],
[ 0, 57, 187, 208, 224, 221, 224, 208, 204, 214, 208, 209, 200,
 159, 245, 193, 206, 223, 255, 255, 221, 234, 221, 211, 220, 232,
 246, 0],
[ 3, 202, 228, 224, 221, 211, 211, 214, 205, 205, 205, 220, 240,
 80, 150, 255, 229, 221, 188, 154, 191, 210, 204, 209, 222, 228,
 225, 0],
[ 98, 233, 198, 210, 222, 229, 229, 234, 249, 220, 194, 215, 217,
 241, 65, 73, 106, 117, 168, 219, 221, 215, 217, 223, 223, 224,
 229, 29],
[ 75, 204, 212, 204, 193, 205, 211, 225, 216, 185, 197, 206, 198,
 213, 240, 195, 227, 245, 239, 223, 218, 212, 209, 222, 220, 221,
 230, 67],
[ 48, 203, 183, 194, 213, 197, 185, 190, 194, 192, 202, 214, 219,
 221, 220, 236, 225, 216, 199, 206, 186, 181, 177, 172, 181, 205,
 206, 115],
[ 0, 122, 219, 193, 179, 171, 183, 196, 204, 210, 213, 207, 211,
 210, 200, 196, 194, 191, 195, 191, 198, 192, 176, 156, 167, 177,
 210, 92],
[ 0, 0, 74, 189, 212, 191, 175, 172, 175, 181, 185, 188, 189,
 188, 193, 198, 204, 209, 210, 210, 211, 188, 188, 194, 192, 216,
 170, 0],
[ 2, 0, 0, 0, 66, 200, 222, 237, 239, 242, 246, 243, 244,
 221, 220, 193, 191, 179, 182, 182, 181, 176, 166, 168, 99, 58,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 40, 61, 44, 72, 41, 35,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0], dtype=uint8)
```

```
[ ] y_train[0]
```

9

```
[ ] class_labels = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

```
[ ] class_labels
```

```
['T-shirt/top',
 'Trouser',
 'Pullover',
 'Dress',
 'Coat',
 'Sandal',
 'Shirt',
 'Sneaker',
 'Bag',
 'Ankle boot']
```

Show image



```
plt.imshow(X_train[0], cmap='Greys')
```

```
plt.figure(figsize=(16,16))

j=1
for i in np.random.randint(0,1000,24):
    plt.subplot(4,6, j);j+=1
    plt.imshow(X_train[i],cmap='Greys')
    plt.axis('off')
    plt.title("{} / {}".format(class_labels[y_train[i]], y_train[i]))
```

```
[ ] #dimension
    X_train.ndim
```

3

```
[ ] X_train = np.expand_dims(X_train,-1)
```

```
▶ X_train.ndim
```

↗ 4

```
[ ] X_test=np.expand_dims(X_test,-1)
```

```
[ ] X_test.ndim
```

4

Features Scaling

```
[ ] X_train = X_train/255
    X_test = X_test/255
```

Step-3:Building the CNN Model:

Split DATASET

```
[ ] from sklearn.model_selection import train_test_split
    X_train,X_validation,y_train,y_validation=train_test_split(X_train,y_train,test_size=0.2,random_state=2020)

[ ] X_train.shape,X_validation.shape,y_train.shape,y_validation.shape

((48000, 28, 28, 1), (12000, 28, 28, 1), (48000,), (12000,))
```

Step-3:Building the CNN model

```
[ ] model=keras.models.Sequential([
    keras.layers.Conv2D(filters=32,kernel_size=3,strides=(1,1),padding='valid',activation='relu',input_shape=[28,28,1]),
    keras.layers.MaxPooling2D(pool_size=(2,2)),
    keras.layers.Flatten(),
    keras.layers.Dense(units=128,activation='relu'),
    keras.layers.Dense(units=10,activation='softmax')
])

[ ] model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 128)	692352
dense_1 (Dense)	(None, 10)	1290
Total params: 693,962		
Trainable params: 693,962		
Non-trainable params: 0		

```
model.compile(optimizer='adam',loss='sparse_categorical_crossentropy',metrics=['accuracy'])

model.fit(X_train,y_train,epochs=10,batch_size=512,verbose=1,validation_data=(X_validation,y_validation))
```

Epoch 1/10
94/94 [=====] - 34s 347ms/step - loss: 0.6398 - accuracy: 0.7817 - val_loss: 0.4346 - val_accuracy: 0.8517
Epoch 2/10
94/94 [=====] - 22s 237ms/step - loss: 0.3893 - accuracy: 0.8634 - val_loss: 0.3642 - val_accuracy: 0.8749
Epoch 3/10
94/94 [=====] - 22s 234ms/step - loss: 0.3318 - accuracy: 0.8847 - val_loss: 0.3325 - val_accuracy: 0.8845
Epoch 4/10
94/94 [=====] - 22s 233ms/step - loss: 0.3033 - accuracy: 0.8930 - val_loss: 0.3152 - val_accuracy: 0.8908
Epoch 5/10
94/94 [=====] - 23s 241ms/step - loss: 0.2812 - accuracy: 0.9009 - val_loss: 0.3079 - val_accuracy: 0.8900
Epoch 6/10
94/94 [=====] - 22s 230ms/step - loss: 0.2644 - accuracy: 0.9064 - val_loss: 0.2985 - val_accuracy: 0.8942
Epoch 7/10
94/94 [=====] - 23s 245ms/step - loss: 0.2481 - accuracy: 0.9105 - val_loss: 0.2864 - val_accuracy: 0.9011
Epoch 8/10
94/94 [=====] - 22s 229ms/step - loss: 0.2348 - accuracy: 0.9164 - val_loss: 0.2973 - val_accuracy: 0.8924
Epoch 9/10
94/94 [=====] - 22s 232ms/step - loss: 0.2230 - accuracy: 0.9207 - val_loss: 0.2795 - val_accuracy: 0.9028
Epoch 10/10
94/94 [=====] - 22s 233ms/step - loss: 0.2144 - accuracy: 0.9235 - val_loss: 0.2702 - val_accuracy: 0.9081
<keras.callbacks.History at 0x7f6bc88cbe90>


```
[ ] y_pred = model.predict(X_test)
y_pred.round(2)

313/313 [=====] - 3s 9ms/step
array([[0. , 0. , 0. , ..., 0.02, 0. , 0.97],
       [0. , 0. , 1. , ..., 0. , 0. , 0. ],
       [0. , 1. , 0. , ..., 0. , 0. , 0. ],
       ...,
       [0. , 0. , 0. , ..., 0. , 0.99, 0. ],
       [0. , 1. , 0. , ..., 0. , 0. , 0. ],
       [0. , 0. , 0.01, ..., 0.19, 0.03, 0. ]], dtype=float32)
```

```
[ ] y_test

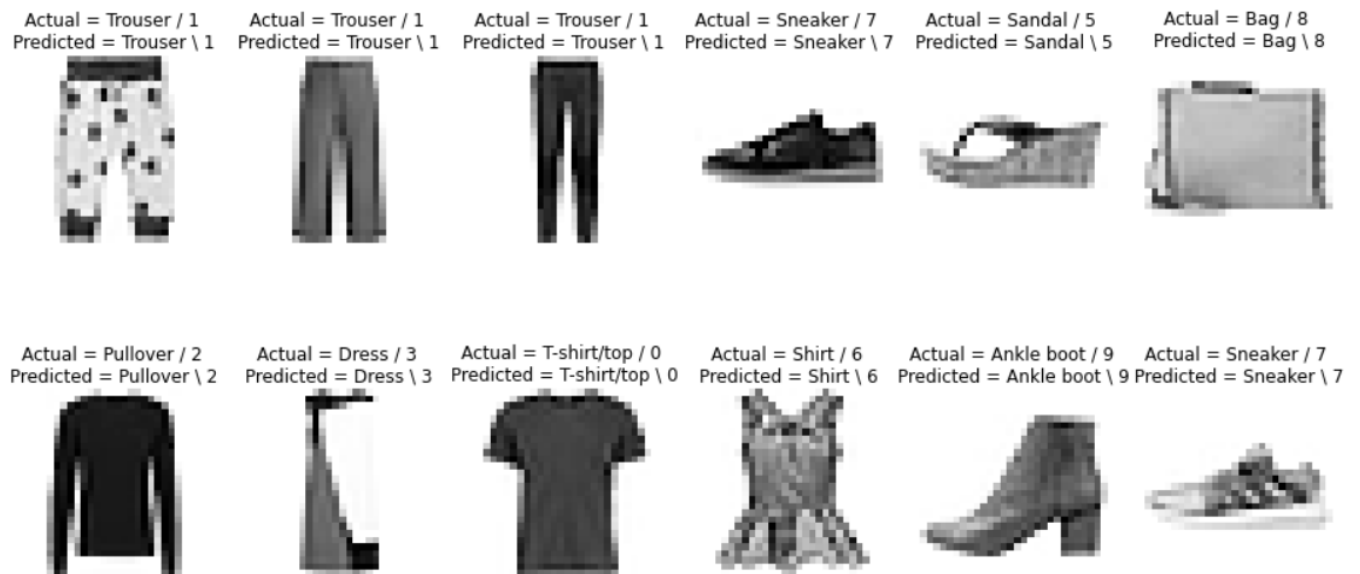
array([9, 2, 1, ..., 8, 1, 5], dtype=uint8)
```

```
[ ] model.evaluate(X_test,y_test)

313/313 [=====] - 3s 9ms/step - loss: 0.2730 - accuracy: 0.9022
[0.2730045020580292, 0.9021999835968018]
```

```
[ ] plt.figure(figsize=(16,16))

j=1
for i in np.random.randint(0,1000,24):
    plt.subplot(4,6, j); j+=1
    plt.imshow(X_test[i].reshape(28,28),cmap='Greys')
    plt.title('Actual = {} / {} \nPredicted = {} \ {}'.format(class_labels[y_test[i]],y_test[i], class_labels[np.argmax(y_pred[i])],np.argmax(y_pred[i])))
    plt.axis('off')
```



Confusion Matrix

```
[ ] from sklearn.metrics import confusion_matrix
plt.figure(figsize=(16,9))
y_pred_labels = [ np.argmax(label) for label in y_pred ]
cm = confusion_matrix(y_test, y_pred_labels)
```

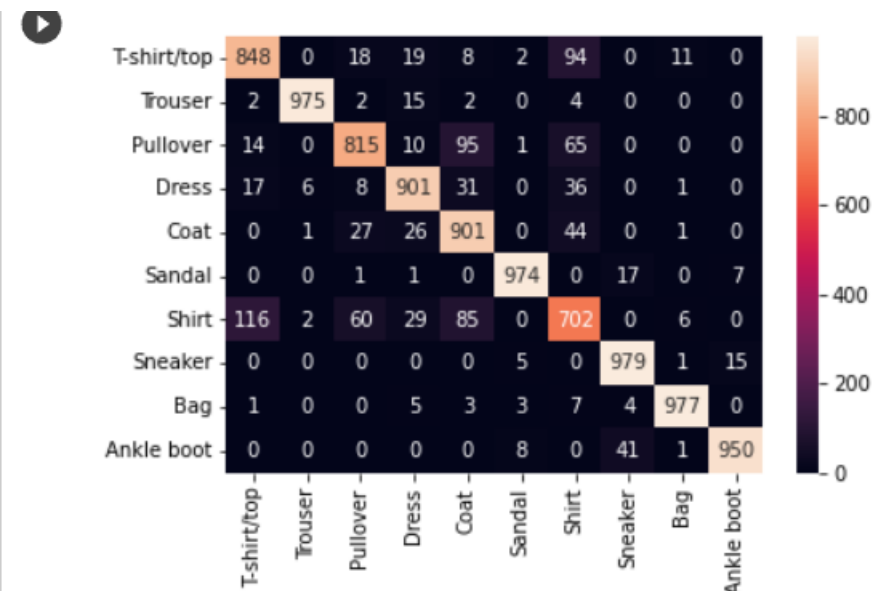
<Figure size 1152x648 with 0 Axes>

```
▶ sns.heatmap(cm, annot=True, fmt='d',xticklabels=class_labels, yticklabels=class_labels)

from sklearn.metrics import classification_report
cr= classification_report(y_test, y_pred_labels, target_names=class_labels)
print(cr)
```

```
↳
```

	precision	recall	f1-score	support
T-shirt/top	0.85	0.85	0.85	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.88	0.81	0.84	1000
Dress	0.90	0.90	0.90	1000
Coat	0.80	0.90	0.85	1000
Sandal	0.98	0.97	0.98	1000
Shirt	0.74	0.70	0.72	1000
Sneaker	0.94	0.98	0.96	1000
Bag	0.98	0.98	0.98	1000
Ankle boot	0.98	0.95	0.96	1000
accuracy			0.90	10000
macro avg	0.90	0.90	0.90	10000
weighted avg	0.90	0.90	0.90	10000



SAVE THE MODEL

```
▶ model.save('AI_INT_CHEREDDY_RAJA_REDDY')
```

6.CONCLUSION:

In this project, i applied various classification methods on an image classification problem. i have explained why the CNNs are the best method we can employ out of considered ones, and why do the other methods fail. Some of the reasons why CNNs are the most practical and usually the most accurate method are:

- 1.They can transfer learning through layers, saving inferences, and making new ones on subsequent layers.
- 2.No need for feature extraction before using the algorithm, it is done during training.
- 3.It recognizes important features.

However, they also have their caveats. They are known to fail on images that are rotated and scaled differently, which is not the case here, as the data was pre-processed. And, although the other methods fail to give that good results on this dataset, they are still used for other tasks related to image processing (sharpening, smoothing etc.).